

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4,BL5)
Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:35

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I D Varshini/192320007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Annexure A

Constraint-Based Problem Solving

Q1. Scenario: Online Banking System

System Components: Account Management, Fund Transfer, Bill Payment, Transaction History

Given Constraints:

- Transaction completion time < 2 seconds
- Multi-factor authentication required
- Support 5000+ concurrent users

Question:

Analyze the system using quality assurance techniques under the given constraints. Identify potential defects related to security, performance, and reliability. Propose suitable testing strategies and justify how they ensure system quality. **(BL4)**

Answer:

The Online Banking System must be tested as a constraint-driven, high-risk application because a defect can affect money, confidential information, and service availability. The quality assurance plan should therefore trace every test to the stated requirements: transactions must complete in less than 2 seconds, multi-factor authentication (MFA) must be enforced, and the system must remain stable with more than 5000 concurrent users.

Constraint Analysis

- 1. Security constraint - MFA:** Every login and sensitive transaction must require the configured second factor. The system should reject missing, invalid, expired, reused, or mismatched verification codes and must not allow users to bypass MFA by directly calling an internal URL or API.
- 2. Performance constraint - transaction time < 2 seconds:** Fund transfers and bill payments should be timed from request submission to final confirmation. The response-time requirement

must be checked under realistic database, network, and concurrent-user conditions rather than only with a single user.

3. Scalability constraint - 5000+ concurrent users: Concurrent sessions should include login, balance enquiry, fund transfer, bill payment, and transaction-history requests. The system must preserve correct balances and acceptable response times while many users access the same services simultaneously.

Potential Defects

Security defects: MFA bypass, weak session invalidation, unauthorized access to another account, repeated acceptance of an OTP, insecure password-reset behavior, insufficient account lockout, injection through input fields, and sensitive information appearing in logs or error messages.

Performance defects: Transactions exceeding 2 seconds, slow database queries, connection-pool exhaustion, CPU or memory saturation, excessive locking, and degradation when the number of active users approaches or exceeds 5000.

Reliability defects: A transfer may debit the sender without crediting the receiver, a retry may create a duplicate transaction, transaction history may not match the final balance, or a server/database failure may leave an operation in an incomplete state.

Testing Strategies and Justification

1. Functional and negative testing: Test valid and invalid account numbers, zero/negative/over-limit amounts, insufficient balance, disabled beneficiaries, incorrect bill details, and unauthorized transactions. This verifies that business rules reject invalid requests before any financial update is committed.

2. Security testing: Execute positive and negative MFA tests, session-expiry tests, authorization tests, repeated-login attempts, input-validation tests, and secure error-message checks. These tests directly confirm that the MFA constraint is enforced and that users cannot gain access beyond their privileges.

3. Load, stress, and spike testing: Generate 5000 or more concurrent virtual users with a realistic mix of banking operations. Measure end-to-end transaction time, throughput, error rate, CPU, memory, database connections, and failed requests. Stress the system beyond the target to identify its safe operating limit. This demonstrates whether the performance and concurrency constraints can be met before deployment.

4. Concurrency and transaction-integrity testing: Run simultaneous transfers involving the same accounts and verify that balances remain correct. Retry timed-out requests and confirm that the same transaction is not posted twice. Database transactions should be checked for atomic commit or rollback so that partial financial updates cannot occur.

5. Recovery and reliability testing: Interrupt a transaction by simulating application-server failure, database failure, network interruption, or service restart. After recovery, verify account balances, transaction status, audit records, and pending operations. This validates fail-safe behavior and continuous reliability.

6. Regression and continuous monitoring: Automate critical login, transfer, payment, and history tests and rerun them after every change. Production monitoring should track response time, failed authentication, transaction failures, and resource utilization so that quality problems are detected early.

A test is accepted only when MFA cannot be bypassed, financial data remains consistent, transactions satisfy the required response-time target under the specified concurrent load, and recovery tests do not create loss or duplication. Thus, the proposed QA strategy covers security, performance, scalability, and reliability while providing measurable evidence against each given constraint.

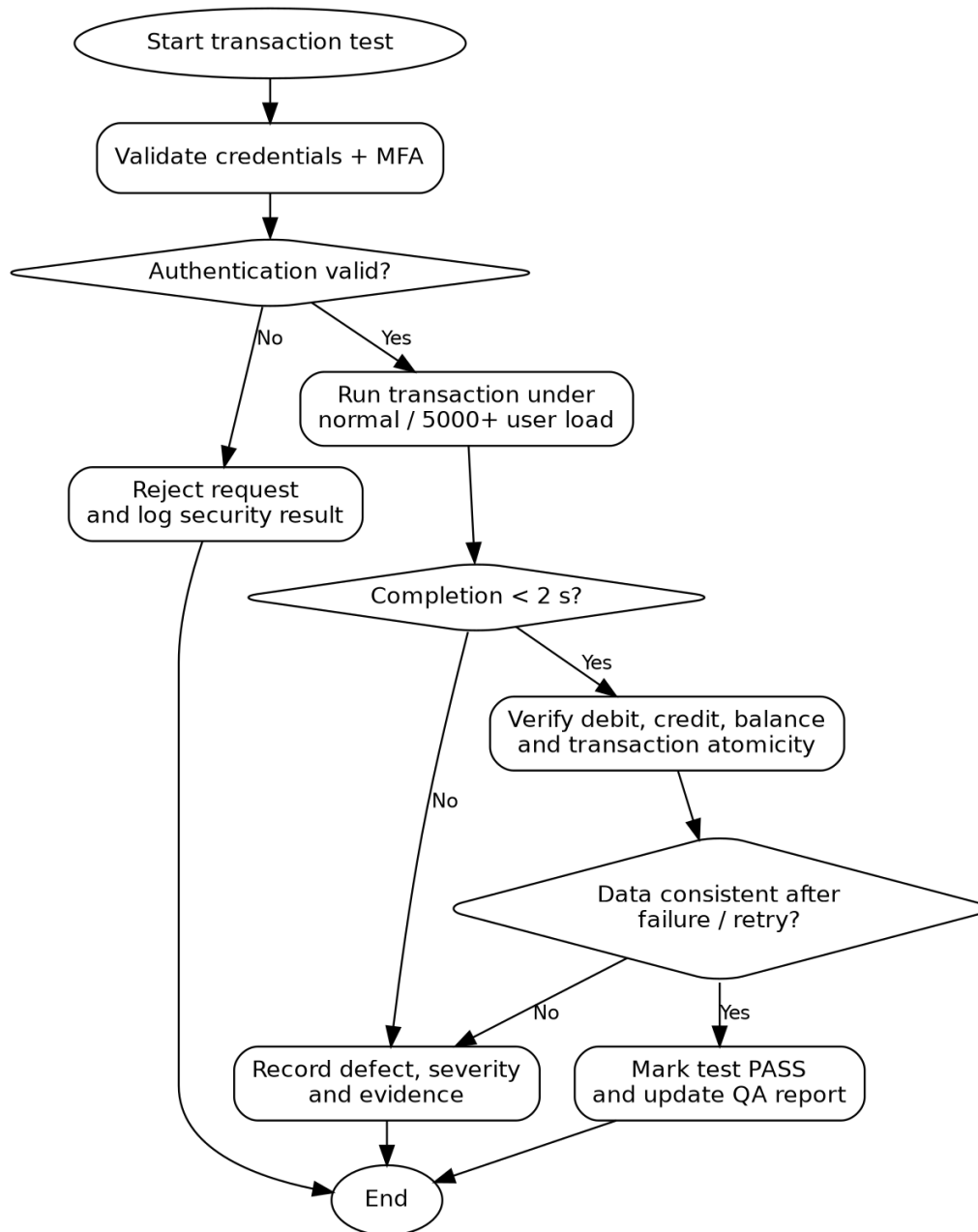


Figure 1. UML activity-style flow for constraint-based banking QA testing

Q2. Scenario: E-Commerce Shopping Application

System Features: Product Search, Shopping Cart, Checkout, Payment Gateway

Usability Constraints:

- Checkout process within 3 steps
- Easy product navigation

- Responsive design for mobile and desktop devices

Question:

Apply usability testing techniques to evaluate the application. Identify usability challenges and recommend design improvements that satisfy the given constraints. Explain how the proposed changes improve customer satisfaction and efficiency. **(BL4)**

Answer:

Usability testing for the E-Commerce Shopping Application should evaluate whether customers can find a product, understand their location in the site, add items to the cart, and complete checkout with minimum effort on both mobile and desktop devices. The evaluation must specifically verify the three stated constraints: checkout within three steps, easy product navigation, and responsive behavior across device sizes.

Usability Challenges

- 1. Excessive checkout effort:** Extra pages, repeated data entry, forced account creation, unclear delivery options, or hidden charges can cause the checkout flow to exceed three meaningful steps and increase abandonment.
- 2. Difficult product navigation:** Poor search relevance, too many categories, weak filters, missing breadcrumbs, inconsistent labels, or an unclear cart icon can make users take unnecessary clicks to locate and purchase a product.
- 3. Mobile and desktop inconsistency:** Layouts may overflow on small screens, touch targets may be too small, text may require zooming, images may load slowly, or desktop navigation may collapse into a confusing mobile menu.
- 4. Error-prone forms and weak feedback:** Users may enter an invalid address or payment detail without receiving immediate guidance. If entered values are cleared after an error, customers must repeat work and may abandon the purchase.

Usability Testing Methods

Task-based user testing: Give participants realistic tasks such as 'find a product under a given price, add it to the cart, and complete checkout.' Include first-time and returning users and test on representative mobile and desktop devices. Record task success, completion time, clicks/taps, errors, backtracking, and abandonment.

Think-aloud observation and post-task feedback: Ask participants to state what they expect while navigating. Confusion at labels, filters, buttons, payment screens, or confirmation messages identifies interface problems that timing data alone may not reveal.

Responsive and cross-device testing: Repeat the same tasks at common screen sizes and orientations. Verify readable text, usable touch targets, correct image scaling, consistent controls, and the absence of horizontal scrolling or overlapping content.

Heuristic and accessibility review: Check consistency, visibility of system status, error prevention, simple wording, keyboard usability, readable contrast, and clear focus order. These checks support a wider range of customers and reduce avoidable interaction errors.

Recommended Design Improvements

1. Enforce a three-step checkout: Keep the cart as the pre-checkout page and organize checkout as Step 1 - delivery address/method, Step 2 - payment, and Step 3 - review and place order. A visible progress indicator should show the current step and allow safe backward navigation without losing data.

2. Improve product discovery: Provide a prominent search bar, relevant suggestions, category navigation, filters, sorting, breadcrumbs, and clearly visible product availability. Preserve selected filters when the user returns from a product page.

3. Use responsive interaction patterns: Adopt flexible layouts, readable text, adequately sized buttons, device-appropriate menus, and optimized images. Primary actions such as Add to Cart and Proceed to Checkout should remain easy to reach on smaller screens.

4. Prevent and recover from errors: Use inline form validation, specific error messages, sensible defaults, address suggestions where appropriate, and preservation of entered data. Show a clear order summary, total cost, payment status, and final confirmation before the session ends.

Evaluation of Improvement

After redesign, repeat the same tasks and compare success rate, checkout completion time, number of interaction steps, navigation errors, abandonment, and user satisfaction ratings. The design is considered improved when customers consistently finish checkout in no more than three steps, reach products with fewer unnecessary actions, and complete the same tasks successfully on both mobile and desktop. These changes reduce cognitive effort and repeated work, which directly improves efficiency and customer satisfaction.

Q3. Scenario: Hospital Management System

Modules: Patient Registration, Appointment Scheduling, Medical Records, Billing

QA Constraints:

- Patient data accuracy must be maintained
- System availability 24/7
- Simultaneous access by multiple departments

Question:

Use quality assurance practices to analyze the system. Identify critical test cases and discuss how the constraints influence testing priorities, defect prevention, and overall system reliability.

(BL5)

Answer:

The Hospital Management System requires strong quality assurance because incorrect or unavailable information can affect patient service and clinical decisions. Testing priorities should be derived from the three constraints: patient data must remain accurate, the system must be available 24/7, and multiple departments must be able to access the system simultaneously without corrupting shared information.

Critical Test Cases

1. Patient registration and data accuracy: Test mandatory fields, valid formats, duplicate patient detection, unique patient identifiers, update of demographics, invalid or incomplete data, and retrieval of the correct patient record. Verify that data saved by one module is displayed consistently in appointments, medical records, and billing.

2. Appointment scheduling: Test available and unavailable doctors, overlapping time slots, duplicate appointments, cancellation, rescheduling, and concurrent attempts to reserve the same slot. The system must prevent conflicting appointments and return a clear status to every requester.

3. Medical-record integrity: Verify that notes, diagnoses, and reports are attached to the correct patient; authorized updates are saved once; older information is not unintentionally overwritten; and an audit trail records important changes. Invalid patient-to-record associations must be rejected.

4. Billing consistency: Check that charges belong to the correct patient and encounter, totals are calculated correctly, duplicate charges are prevented, and billing remains synchronized with completed services. Failed updates must not create partially saved financial records.

5. Concurrent departmental access: Simulate registration, nursing, doctor, laboratory, pharmacy, and billing users reading and updating shared records at the same time. Verify transaction isolation, locking or version-control behavior, conflict messages, and final database consistency.

6. Availability and recovery: Run long-duration tests and simulate application restart, database failover, network interruption, backup restoration, and service recovery. Confirm that users can resume operation and that no committed patient data is lost or duplicated.

How the Constraints Influence Testing Priorities

Patient data accuracy has the highest functional priority: Validation, referential integrity, duplicate prevention, transaction correctness, and auditability should be tested before convenience features. Defects that could associate information with the wrong patient should be classified as critical.

24/7 availability increases reliability priority: The QA plan must include failover, recovery-time, backup, restart, and endurance testing rather than relying only on normal functional tests. Planned maintenance and component failures must not leave data in an unknown state.

Simultaneous access increases concurrency priority: Tests must deliberately create conflicting updates and heavy read/write activity. A system that works for one user can still fail when several departments update related records together, so concurrency tests are essential.

Defect Prevention Practices

Defects can be prevented by reviewing requirements with hospital stakeholders, defining validation rules before coding, using database constraints for mandatory and unique values, applying controlled transactions to related updates, reviewing code, and automating regression tests for critical workflows. A staging environment should reproduce realistic data volume and departmental concurrency. Monitoring and audit logging should then detect unusual failures, slow operations, and data-integrity events in production.

Overall system reliability is achieved when the same patient information remains correct across modules, conflicting operations are handled predictably, the service recovers cleanly from infrastructure failures, and the most safety-critical workflows remain available continuously.

Therefore, the given constraints do not merely add test cases; they determine the order, severity, and depth of the entire QA process.

Q4. Scenario: Smart Library Management System

Features: Book Search, Book Reservation, Borrow/Return, Notifications

Usability Constraints:

- Search results displayed within 2 seconds
- Accessible interface for all age groups
- Minimal learning curve for first-time users

Question:

Conduct a usability evaluation using appropriate testing methods. Analyze how the given constraints impact interface design and recommend improvements to enhance usability and accessibility. **(BL5)**

Answer:

The Smart Library Management System should be evaluated with both observed user tasks and measurable performance checks. The usability plan must verify that search results appear within 2 seconds, users of different age groups can understand and operate the interface, and a first-time user can complete common tasks with very little learning.

Usability Evaluation Methods

1. Task-based usability testing: Ask participants to search for a book, filter results, reserve an available copy, check borrowing status, return a book, and locate a notification. Include first-time users and participants from different age groups. Measure task success, completion time, errors, help requests, unnecessary clicks, and backtracking.

2. Search-performance testing: Measure the time from submitting a search to displaying usable results with small and large catalogs, common and rare keywords, filters, and concurrent requests. The interface should meet the stated 2-second requirement under representative conditions.

3. First-use learnability testing: Give new users common tasks without detailed instructions. Observe whether labels such as Search, Reserve, Borrowed Books, Return, and Notifications are self-explanatory. Repeated hesitation or dependence on help indicates an excessive learning curve.

4. Accessibility and heuristic evaluation: Review keyboard navigation, focus visibility, readable text size, contrast, meaningful labels, predictable navigation, screen-reader-friendly controls, clear error messages, and consistent button placement. This supports users with different ages, abilities, and levels of technical experience.

Impact of Constraints on Interface Design

Search within 2 seconds: The search page should provide immediate feedback that a request is being processed, avoid unnecessarily heavy visual elements, and present the most useful information first. From the technical side, efficient indexing, pagination, and caching of safe search data can help maintain the target without changing the user's task.

Accessible for all age groups: The interface should use simple terminology, readable typography, strong visual hierarchy, sufficiently large controls, and avoid relying only on color to convey status. Important functions should not be hidden behind unfamiliar icons.

Minimal learning curve: Core actions should remain in predictable locations, use consistent names, and require as few screens as practical. First-time users should understand what to do from the interface itself rather than from a separate manual.

Recommended Improvements

1. Simplify the home and search screens: Place a prominent search field on the first screen, provide common filters such as title, author, category, and availability, and show clear result cards with availability and a direct Reserve action.

2. Make navigation predictable: Use a small set of clearly named destinations such as Home, Search, My Books, Reservations, and Notifications. Show breadcrumbs or a clear back path where users can move into deeper screens.

3. Improve readability and accessibility: Use scalable text, sufficient spacing, visible focus indicators, keyboard support, descriptive labels, text alternatives for meaningful images, and understandable status messages such as 'Reserved until 5:00 PM' rather than ambiguous symbols.

4. Provide clear feedback and error recovery: After reservation, borrowing, or return actions, display an immediate confirmation and updated status. If a book is unavailable, explain why

and provide alternatives such as a waiting list or similar titles instead of leaving the user at a dead end.

5. Support first-time users without clutter: Use short contextual hints for unfamiliar features and allow help to be opened when required, while keeping the normal workflow simple for experienced users.

Evaluation Result

The redesigned interface should be retested with the same tasks. Improvement is demonstrated when search results consistently meet the 2-second target, task success increases, navigation errors and help requests decrease, and first-time users complete key tasks independently. Because the recommendations directly map to speed, accessibility, and learnability, they improve both usability and the practical reach of the library service.

Q5. Scenario: Airline Reservation System

Modules: User Login, Flight Search, Seat Selection, Ticket Booking

QA & Usability Constraints:

- No booking duplication
- System must support peak holiday traffic
- Booking interface should be simple and error-free

Question:

Apply both quality assurance and usability testing techniques under the specified constraints. Identify major risks, propose suitable testing methods, and explain how the system can be optimized for reliability, performance, and user experience. **(BL4)**

Answer:

The Airline Reservation System combines a high-concurrency transactional process with a customer-facing interface. The QA and usability strategy must therefore protect booking correctness during peak traffic while keeping the booking flow simple enough that users do not accidentally choose the wrong flight, seat, or payment option.

Major Risks

- 1. Duplicate or conflicting booking:** Two users may attempt to reserve the same seat at nearly the same time, or one user may retry after a timeout and unintentionally create a second booking. This is the most important correctness risk because the specification explicitly requires no booking duplication.
- 2. Peak holiday overload:** Flight search, seat maps, login, and booking requests may become slow or fail when traffic rises suddenly. Resource exhaustion can also cause timeouts that make customers repeat actions and increase the chance of inconsistent transactions.
- 3. Payment-booking inconsistency:** Payment may succeed while ticket creation fails, or a seat may remain blocked after payment failure. Such partial operations create customer complaints and require difficult manual correction.
- 4. Stale seat availability:** A seat displayed as available may already have been selected by another user. If the interface does not refresh or explain the conflict clearly, users may believe the system has lost their booking.
- 5. Usability errors:** Complex forms, unclear fare rules, hidden validation errors, confusing seat selection, or weak confirmation can lead users to abandon the task or submit incorrect passenger information.

Quality Assurance Testing Methods

Functional and boundary testing: Test valid and invalid logins, route/date combinations, passenger data, seat choices, fare conditions, payment outcomes, and booking confirmation. Confirm that mandatory information is validated before the booking is committed.

Concurrency and duplicate-booking testing: Send simultaneous requests for the same flight and seat from many sessions. Only one request should obtain the seat; all others must receive an accurate unavailable response. Retry the same booking request after timeouts and verify that idempotent or unique transaction controls prevent a duplicate record.

Integration and failure testing: Simulate successful payment, payment rejection, delayed gateway response, network loss after payment, booking-service failure, and notification failure. Verify that seat holds are released when required and that confirmed payments are reconciled with exactly one booking.

Load, stress, spike, and endurance testing: Model peak holiday traffic with large numbers of users searching, viewing seat maps, and booking simultaneously. Measure response time, throughput, error rate, database load, queue length, and resource utilization. Spike tests verify

sudden surges, while endurance tests reveal memory leaks and performance degradation over time.

Recovery and regression testing: Restart services or fail over infrastructure during active bookings and confirm that committed bookings remain intact and incomplete seat holds expire safely. Automate critical booking tests so that changes to fares, payment, or seat-allocation logic do not reintroduce duplication defects.

Usability Testing Methods

Observe representative users completing a full task: login, search for a flight, compare options, choose a seat, enter passenger details, pay, and locate the final ticket. Record completion rate, time on task, errors, backtracking, abandoned steps, and points of hesitation. Test on mobile and desktop and verify that validation messages are specific, the selected flight and fare remain visible, seat status is understandable, and final confirmation clearly states the booking reference and passenger details.

Optimization Recommendations

1. Protect seat allocation: Use an atomic seat-reservation operation with a short temporary hold, a unique constraint for flight-seat-booking combinations, and an idempotent booking request identifier. These controls ensure that concurrency or retries cannot create two valid bookings for one seat.

2. Prepare for peak traffic: Use load balancing and scalable application instances, optimize database queries, cache safe read-heavy data such as common flight-search results, and isolate critical booking writes from nonessential work. Queue noncritical activities such as some notifications so that they do not delay the booking transaction.

3. Maintain transaction consistency: Coordinate seat hold, payment status, and booking confirmation so that every failure has a defined recovery action. Time-limited holds should be released automatically, and reconciliation should detect any payment that does not have the expected booking result.

4. Simplify the booking interface: Use a clear step sequence, retain entered information when validation fails, show fare and baggage details before payment, prevent impossible selections, and present concise inline error messages. The final screen should provide an unmistakable success status and booking reference.

The system is optimized when concurrent requests cannot duplicate a seat, peak-load tests remain within agreed performance and error-rate limits, failed integrations recover without

inconsistent payment or booking states, and users can complete the booking with few errors or unnecessary steps. Combining QA and usability testing is essential because technical correctness alone does not prevent user mistakes, while a simple interface alone cannot guarantee reliable seat allocation under heavy traffic.

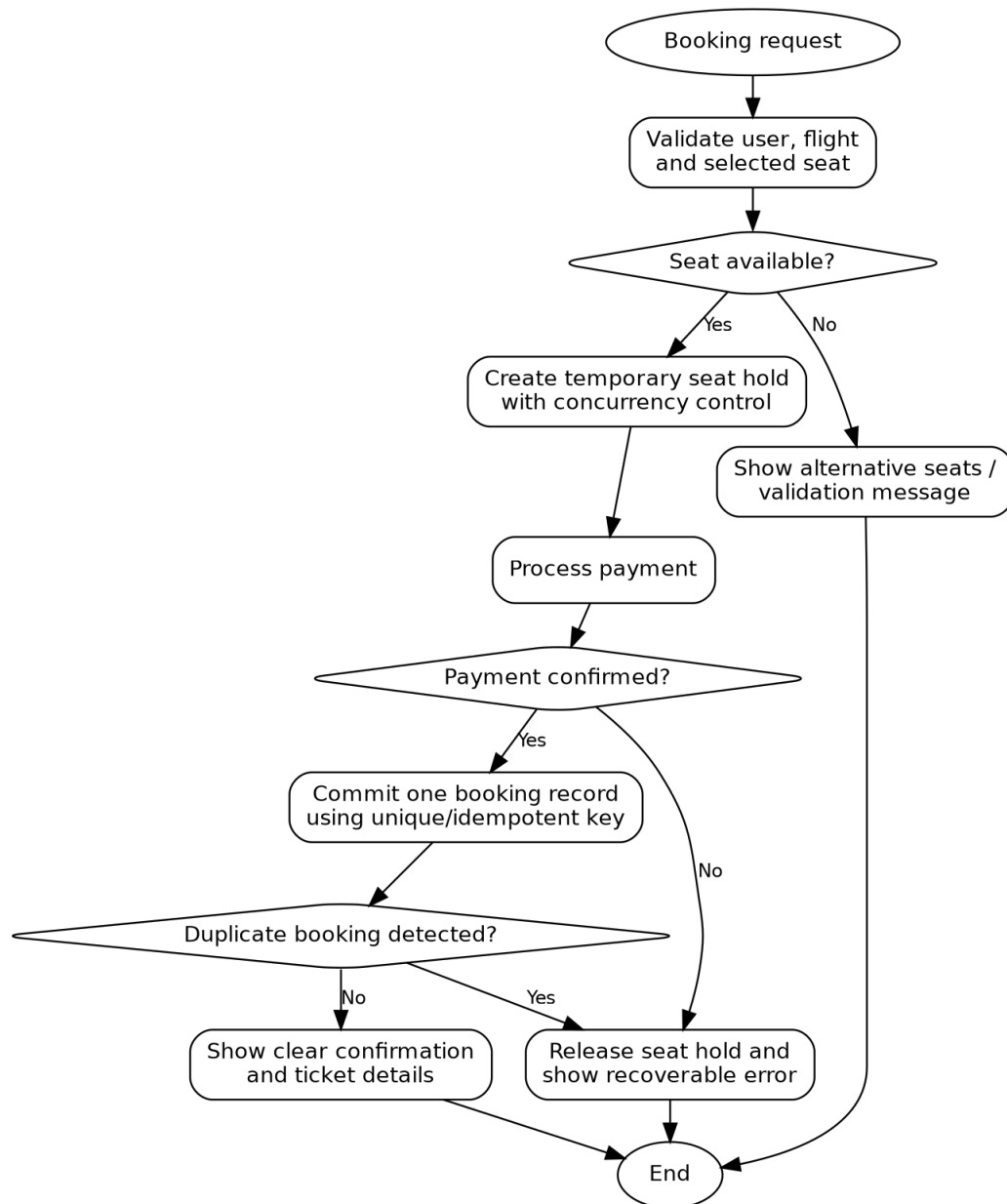


Figure 2. UML activity-style flow for duplicate-safe airline booking